



## SOFTWARE REQUIREMENTS SPECIFICATION

# FFIRE

## Financial Fraud Investigation Reasoning Engine

An Explainable Multi-Step Reasoning Platform for Enterprise Financial  
Fraud Investigation using LangGraph and Large Language Models

---

**Document Version:** 1.0 — Initial Release

**Date:** July 2026

**Classification:** Confidential — Internal Engineering

**Platform:** LangGraph · FastAPI · PostgreSQL · Vector DB · Claude API

Designed for enterprise fintech environments where correctness, transparency, and auditability are as important as generating an answer.

# Table of Contents

---

|            |                             |     |
|------------|-----------------------------|-----|
| <b>01.</b> | Introduction                | ... |
| <b>02.</b> | Literature Review           | ... |
| <b>03.</b> | Problem Statement           | ... |
| <b>04.</b> | System Objectives           | ... |
| <b>05.</b> | Overall System Architecture | ... |
| <b>06.</b> | LangGraph Execution Graph   | ... |
| <b>07.</b> | Detailed Module Design      | ... |
| <b>08.</b> | Data Flow Diagrams          | ... |
| <b>09.</b> | Entity Relationship Diagram | ... |
| <b>10.</b> | Database Schema             | ... |
| <b>11.</b> | Sequence Diagrams           | ... |
| <b>12.</b> | Use Case Diagram            | ... |
| <b>13.</b> | State Machine Diagram       | ... |
| <b>14.</b> | Activity Diagram            | ... |
| <b>15.</b> | Component Diagram           | ... |
| <b>16.</b> | Deployment Diagram          | ... |
| <b>17.</b> | Security Architecture       | ... |
| <b>18.</b> | Reasoning Strategy          | ... |
| <b>19.</b> | Prompt Engineering          | ... |
| <b>20.</b> | Guardrails                  | ... |
| <b>21.</b> | Risk Scoring                | ... |
| <b>22.</b> | API Specification           | ... |
| <b>23.</b> | Frontend Design             | ... |
| <b>24.</b> | Future Scope                | ... |
| <b>25.</b> | Conclusion                  | ... |



# Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) defines the functional, non-functional, and architectural requirements for the **Financial Fraud Investigation Reasoning Engine (FFIRE)** — a multi-step, explainable reasoning platform that investigates suspicious financial transactions using a deterministic LangGraph-based execution pipeline, structured data retrieval, and Large Language Model (LLM) reasoning grounded in verifiable evidence. This document is intended for engineering teams, architects, QA, compliance stakeholders, and reviewers who require a precise, unambiguous specification of system behaviour.

## 1.2 Scope

FFIRE investigates individual flagged transactions and produces a fully traceable, evidence-backed investigation report explaining *why* a transaction is suspicious — rather than emitting an opaque risk score. The system decomposes an investigation into discrete reasoning tasks (customer history, merchant reputation, device fingerprint, geolocation, transaction velocity, and historical fraud cases), retrieves supporting evidence from structured and unstructured data sources, applies a hybrid rule-based and LLM-based risk reasoning layer, validates conclusions against guardrails to prevent hallucination, and produces a final report suitable for compliance audit and human review.

### In Scope

Transaction-level fraud investigation, evidence retrieval orchestration, explainable risk reasoning, retry and escalation logic, audit logging, REST API, investigator dashboard.

### Out of Scope

Real-time transaction blocking, core banking ledger modification, payment processing, and regulatory filing automation (e.g. SAR submission) are out of scope for this release; the engine produces recommendations only.

## 1.3 Objectives

### Explainability

Every conclusion must be traceable to retrieved evidence — no unsupported claims are permitted in a final report.

### Determinism

Identical inputs must always traverse the same execution graph, ensuring reproducible, auditable investigations.

### Resilience

Transient retrieval failures must be retried automatically before escalating to human review.

### Auditability

Every state transition, tool call, and evidence reference is logged for compliance review.

## 1.4 Background

Traditional fraud detection systems rely on supervised machine learning models or static rule engines that output a binary flag or a numeric risk score with little to no explanation. Investigators must then manually reconstruct the supporting evidence — a process that can take hours per case. FFIRE inverts this model: instead of predicting fraud and asking a human to justify it, the system performs the investigation itself, in the same structured manner a human financial crime analyst would, and presents the supporting evidence alongside the conclusion.

## 1.5 Existing Problems

| Problem                     | Impact                                                       |
|-----------------------------|--------------------------------------------------------------|
| Black-box ML risk scores    | Analysts cannot justify decisions to regulators or customers |
| High false-positive rates   | Wasted investigator hours, customer friction                 |
| Free-form LLM chat agents   | Inconsistent reasoning paths, hallucinated conclusions       |
| Manual evidence gathering   | Investigations take hours instead of seconds                 |
| No audit trail of reasoning | Difficult to pass regulatory and internal audit review       |

## 1.6 Proposed Solution

FFIRE implements the investigation as an explicit, inspectable **LangGraph state machine**. Each node performs one well-defined responsibility (planning, retrieval, comparison, reasoning, validation) and updates a shared, strongly-typed state object. Guardrail nodes enforce that every claim in the final report is grounded in retrieved evidence, and a confidence-based routing mechanism escalates uncertain cases to human review rather than guessing.

## 1.7 Expected Benefits

- Investigation turnaround reduced from hours to seconds for routine cases.
- Every conclusion is fully traceable to source evidence, satisfying audit and regulatory requirements.
- Deterministic execution graph enables reliable testing, monitoring, and continuous improvement.

- Human-in-the-loop escalation preserves analyst oversight for high-risk or ambiguous cases.

# Literature Review

## 2.1 Rule-Based Fraud Detection

Early fraud detection systems relied on hand-crafted rules (e.g. transaction amount thresholds, velocity limits). These systems are interpretable but brittle — they cannot adapt to novel fraud patterns and generate high false-positive rates as fraud tactics evolve.

## 2.2 Machine Learning Approaches

Supervised models (gradient boosted trees, neural networks) trained on historical labelled fraud data improve detection accuracy but sacrifice interpretability. Post-hoc explanation methods (SHAP, LIME) approximate feature importance but do not constitute a verifiable, evidence-grounded explanation.

## 2.3 Explainable AI (XAI)

Explainable AI research emphasizes that high-stakes financial decisions require explanations that are faithful to the underlying decision process, not merely plausible-sounding narratives. FFIRE addresses this by constraining the LLM reasoning layer to cite only retrieved, verifiable evidence.

## 2.4 Multi-Agent and Graph-Orchestrated Systems

Recent advances in agentic AI frameworks (LangGraph, AutoGen, CrewAI) allow developers to define explicit execution graphs rather than relying on free-form agent loops. This provides control over reasoning order, supports retries and conditional branching, and produces deterministic, auditable execution traces.

## 2.5 Retrieval-Augmented Generation (RAG)

RAG grounds LLM outputs in retrieved documents rather than parametric memory, substantially reducing hallucination. FFIRE extends this concept beyond document retrieval to structured database queries, API calls, and historical case retrieval from a vector knowledge base.

## 2.6 Enterprise AI Requirements

Enterprise deployments — particularly in regulated industries such as banking — demand reproducibility, audit trails, role-based access control, and human oversight. These requirements directly shape FFIRE's architecture, as detailed in subsequent chapters.



# Problem Statement

## 3.1 Statement

Existing financial fraud detection systems output risk scores or binary flags without a transparent, verifiable chain of reasoning, forcing human investigators to manually reconstruct evidence. Free-form conversational AI agents, while more flexible, lack deterministic execution and are prone to hallucinated or unsupported conclusions — both unacceptable in a regulated financial environment.

## 3.2 Why Current Systems Fail

| Failure Mode              | Description                                                                                              |
|---------------------------|----------------------------------------------------------------------------------------------------------|
| False Positives           | Static rule thresholds flag legitimate transactions, eroding investigator trust and customer experience. |
| Hallucination             | Free-form LLM agents may fabricate plausible-sounding but unverified explanations.                       |
| No Explainability         | ML risk scores cannot be traced back to specific evidentiary facts required for compliance review.       |
| No Investigation Workflow | Systems flag transactions but do not perform the multi-step investigation a human analyst would.         |

## 3.3 Design Implication

FFIRE must combine the flexibility of LLM reasoning with the determinism, auditability, and grounding guarantees of a structured workflow engine — the central architectural decision explored in Chapters 5–9.

# System Objectives

## 4.1 Functional Objectives

- FO-1: Accept an investigation request for a transaction or customer identifier.
- FO-2: Decompose the investigation into discrete, ordered reasoning tasks.
- FO-3: Retrieve customer, transaction, merchant, device, and location evidence in parallel.
- FO-4: Compare retrieved evidence against historical fraud patterns in the knowledge base.
- FO-5: Compute a combined risk score using rule-based and LLM-based reasoning.
- FO-6: Validate that every claim in the generated report is grounded in retrieved evidence.
- FO-7: Retry failed retrieval or validation steps up to a configurable threshold.
- FO-8: Escalate to human review when confidence falls below a configurable threshold.
- FO-9: Generate a final, structured, human-readable investigation report.
- FO-10: Log every node execution, tool call, and decision for audit purposes.

## 4.2 Non-Functional Objectives

| Category        | Requirement                                                                   |
|-----------------|-------------------------------------------------------------------------------|
| Performance     | P95 investigation latency < 8 seconds for standard-complexity cases.          |
| Availability    | 99.9% uptime for the API gateway and orchestration layer.                     |
| Scalability     | Support horizontal scaling of retrieval and reasoning workers via Kubernetes. |
| Security        | All PII encrypted at rest (AES-256) and in transit (TLS 1.2+).                |
| Auditability    | Every investigation must produce an immutable, timestamped audit trail.       |
| Explainability  | 100% of generated claims must cite a specific evidence source.                |
| Maintainability | Graph nodes must be independently testable and replaceable.                   |

## 4.3 Business Objectives

- Reduce average fraud investigation time by at least 80%.
- Reduce false-positive escalations to human analysts by at least 30%.
- Provide audit-ready documentation for every automated decision.

## 4.4 Research Objectives

- Evaluate deterministic graph-based orchestration against free-form agentic reasoning for grounding quality.

- Measure the effectiveness of confidence-threshold-based human escalation in reducing investigator workload without increasing missed-fraud rates.

# Overall System Architecture

## 5.1 Architectural Overview

FFIRE follows a layered architecture: a presentation layer (investigator dashboard), an API gateway, a LangGraph orchestration layer, a retrieval layer spanning structured and unstructured data sources, and a reasoning/validation layer that produces the final explainable report.

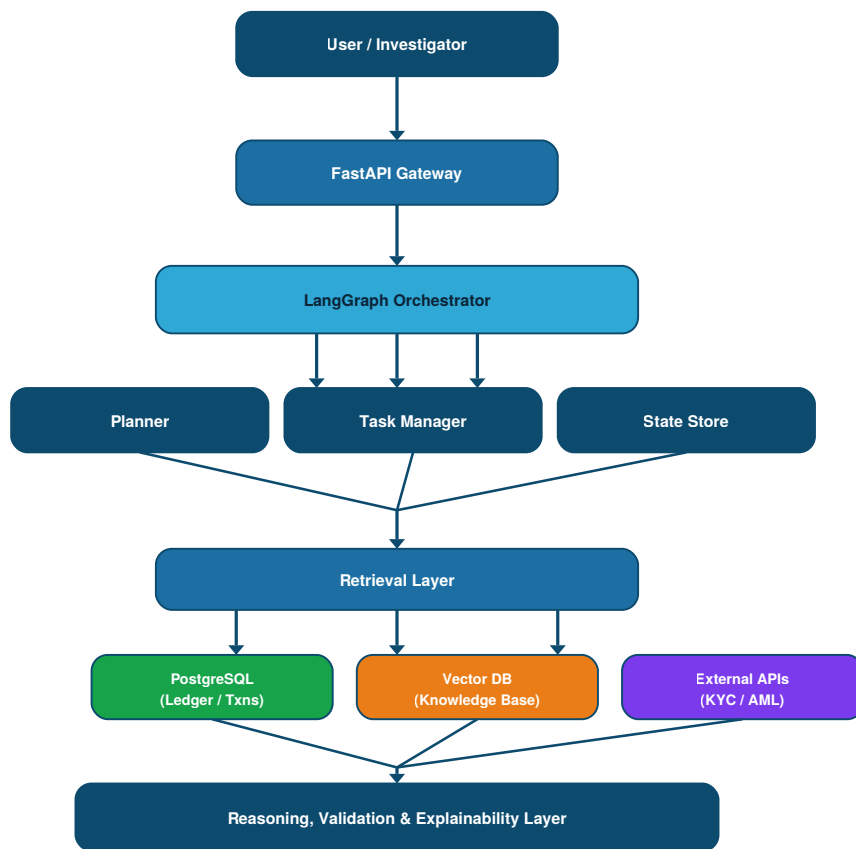


Figure 5.1 — FFire High-Level System Architecture

## 5.2 Layer Responsibilities

| Layer                  | Responsibility                                                               |
|------------------------|------------------------------------------------------------------------------|
| API Gateway            | Authentication, request validation, rate limiting, routing to orchestrator.  |
| LangGraph Orchestrator | Executes the deterministic investigation state machine.                      |
| Retrieval Layer        | Queries PostgreSQL ledger, vector knowledge base, and external KYC/AML APIs. |
| Reasoning Layer        | Combines rule engine output with LLM-based grounded reasoning.               |

|                      |                                                                    |
|----------------------|--------------------------------------------------------------------|
| Validation Layer     | Enforces guardrails; rejects unsupported claims; triggers retries. |
| Explainability Layer | Assembles the final report with inline evidence citations.         |

# LangGraph Execution Graph

## 6.1 Execution Graph

The investigation is modelled as an explicit directed graph of nodes, each performing a single responsibility and reading/writing a shared state object. The graph guarantees that the AI cannot skip steps or reorder reasoning arbitrarily.

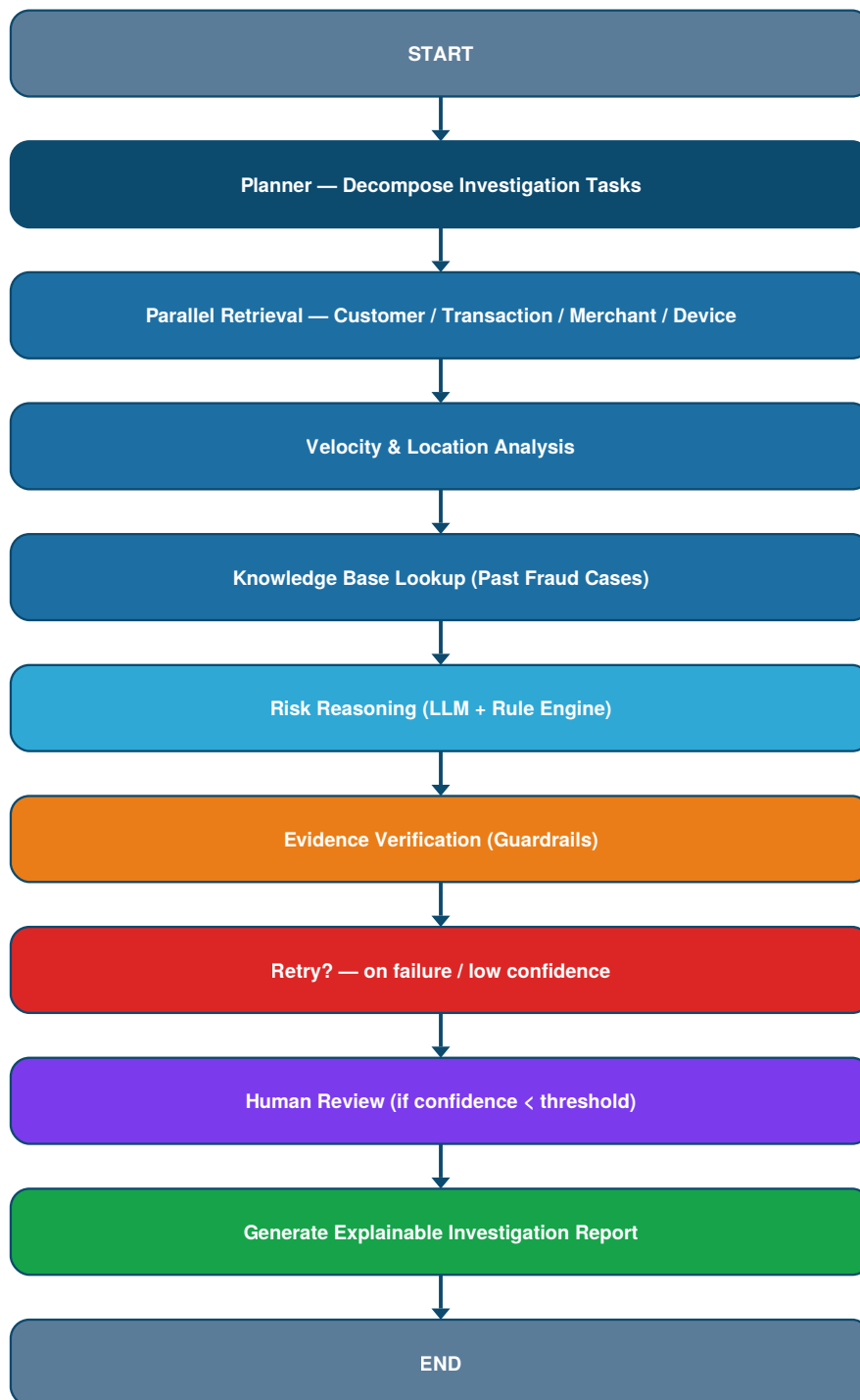


Figure 6.1 — FFIRE LangGraph Execution Pipeline

## 6.2 Shared State Object

```
state = {
  "transaction_id": "T-78491",
  "tasks": [],
  "customer_evidence": {},
  "transaction_evidence": {},
  "merchant_evidence": {},
  "device_evidence": {},
  "location_evidence": {},
  "historical_cases": [],
  "risk_score": None,
  "confidence": None,
  "validated": False,
  "retry_count": 0,
  "report": None
}
```

## 6.3 Node Definitions (LangGraph)

```
graph.add_node("planner", planner_node)
graph.add_node("retrieve_customer", retrieve_customer_node)
graph.add_node("retrieve_transaction", retrieve_transaction_node)
graph.add_node("retrieve_merchant", retrieve_merchant_node)
graph.add_node("retrieve_device", retrieve_device_node)
graph.add_node("velocity_check", velocity_node)
graph.add_node("knowledge_lookup", knowledge_node)
graph.add_node("risk_reasoning", reasoning_node)
graph.add_node("validator", validator_node)
graph.add_node("human_review", human_review_node)
graph.add_node("report_generator", report_node)
```



# Detailed Module Design

## 7.1 Planner Module

Decomposes the incoming investigation request into an ordered list of evidence-gathering and analysis tasks based on the transaction type and available data sources.

## 7.2 Retriever Modules

| Module                | Data Source                   | Output                                   |
|-----------------------|-------------------------------|------------------------------------------|
| Customer Retriever    | PostgreSQL — customer_profile | KYC status, account age, risk tier       |
| Transaction Retriever | PostgreSQL — transactions     | Amount, timestamp, channel, status       |
| Merchant Retriever    | PostgreSQL / External API     | Merchant category, historical fraud rate |
| Device Retriever      | Device fingerprint service    | Device ID, OS, first-seen date           |
| Location Retriever    | Geo-IP / GPS service          | Country, distance from home location     |

## 7.3 Tool Executor

A unified interface that wraps all external tool calls (database queries, API calls, vector search) with timeout handling, retry logic, and structured error reporting back into the shared state object.

## 7.4 Fraud Analyzer

Applies a hybrid rule engine (velocity limits, geolocation mismatch, device-change heuristics) combined with LLM-based pattern reasoning over the aggregated evidence.

## 7.5 Knowledge Retriever

Performs a similarity search over a vector database of previously confirmed fraud cases to surface analogous historical patterns relevant to the current investigation.

## 7.6 Evidence Collector

Aggregates outputs from all retriever modules into a normalized evidence bundle consumed by the reasoning engine.

## 7.7 Reasoning Engine

Constructs a grounded prompt containing only retrieved evidence (no free recall) and instructs the LLM to produce a structured risk explanation with explicit citations to evidence fields.

## **7.8 Validator**

Parses the LLM output and verifies that every factual claim references a field present in the evidence bundle; unsupported claims trigger a retry or escalation.

## **7.9 Confidence Calculator**

Combines rule-engine score, LLM self-reported confidence, and evidence completeness into a single normalized confidence value used for routing decisions.

## **7.10 Audit Logger**

Persists every node execution, input/output state delta, tool call, and timing metric to an immutable audit log table for compliance review.

## **7.11 Human Review Module**

Pauses graph execution and surfaces the case, evidence, and draft explanation to a human analyst when confidence is below threshold or a high-risk action is detected.

# Data Flow Diagrams

## 8.1 DFD Level 0 — Context Diagram

The context diagram shows FFIRE as a single process interacting with external entities: the fraud analyst, core banking system, knowledge base, and compliance/audit function.

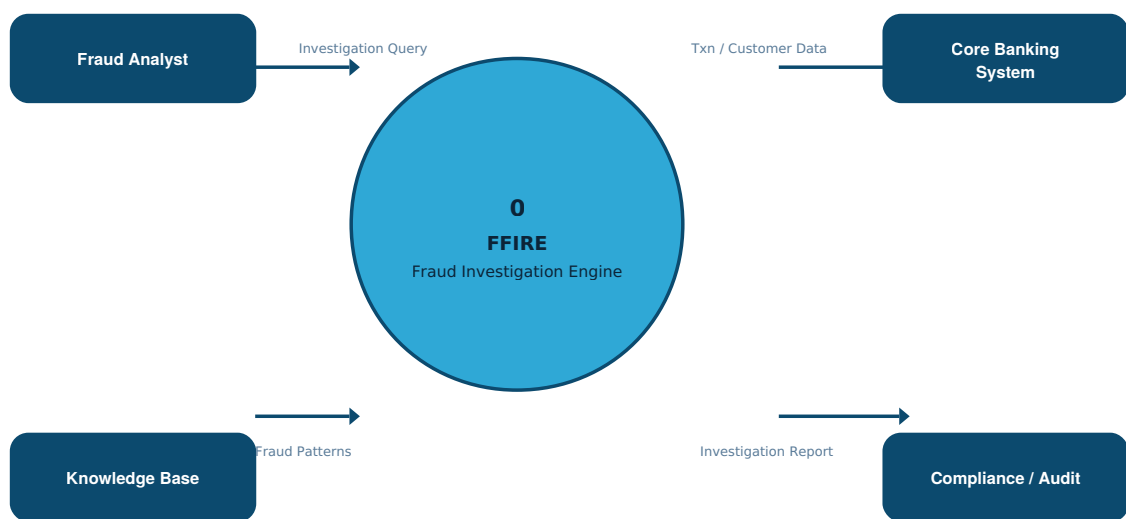


Figure 8.1 — DFD Level 0 (Context Diagram)

## 8.2 DFD Level 1 — Process Decomposition

Level 1 decomposes FFIRE into six major processes and three persistent data stores.

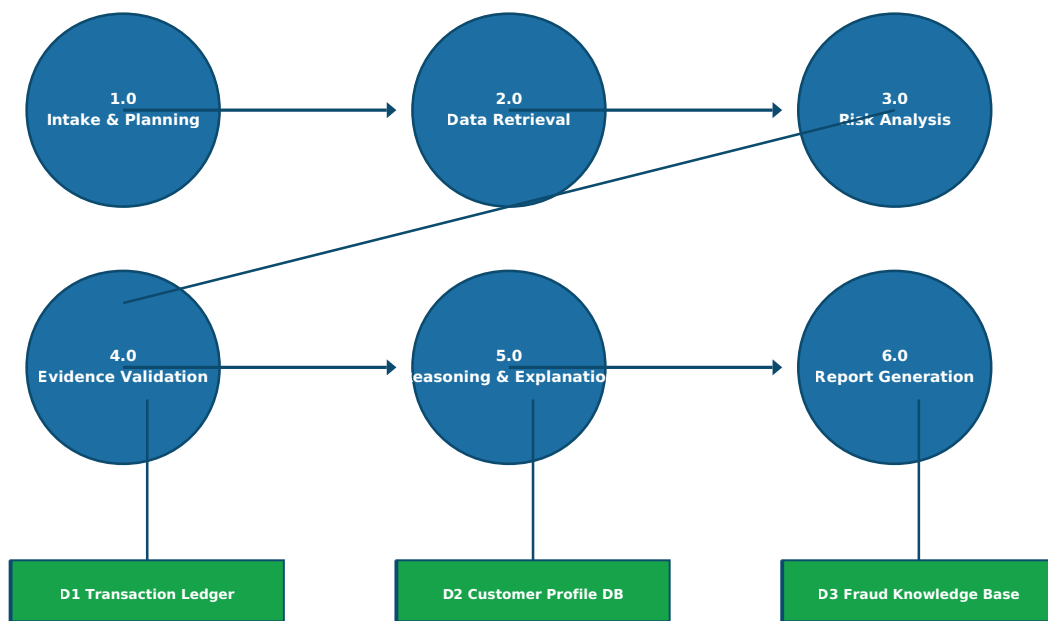


Figure 8.2 — DFD Level 1

### 8.3 DFD Level 2 — Risk Analysis Process (Narrative)

Process 3.0 (Risk Analysis) further decomposes into: 3.1 Velocity Check, 3.2 Geolocation Comparison, 3.3 Device Trust Scoring, 3.4 Merchant Risk Lookup, and 3.5 Historical Pattern Matching against the fraud knowledge base. Each sub-process reads from the evidence bundle produced in Level 1 and writes a partial risk signal that is aggregated by the Confidence Calculator before reasoning proceeds.

# Entity Relationship Diagram

## 9.1 Entities

The FFIRE data model captures twelve core entities spanning identity, transactional, evidentiary, and audit domains: User, Customer, Account, Transaction, Merchant, Device, Location, Investigation, Evidence, FraudCase, RiskScore, and AuditLog.

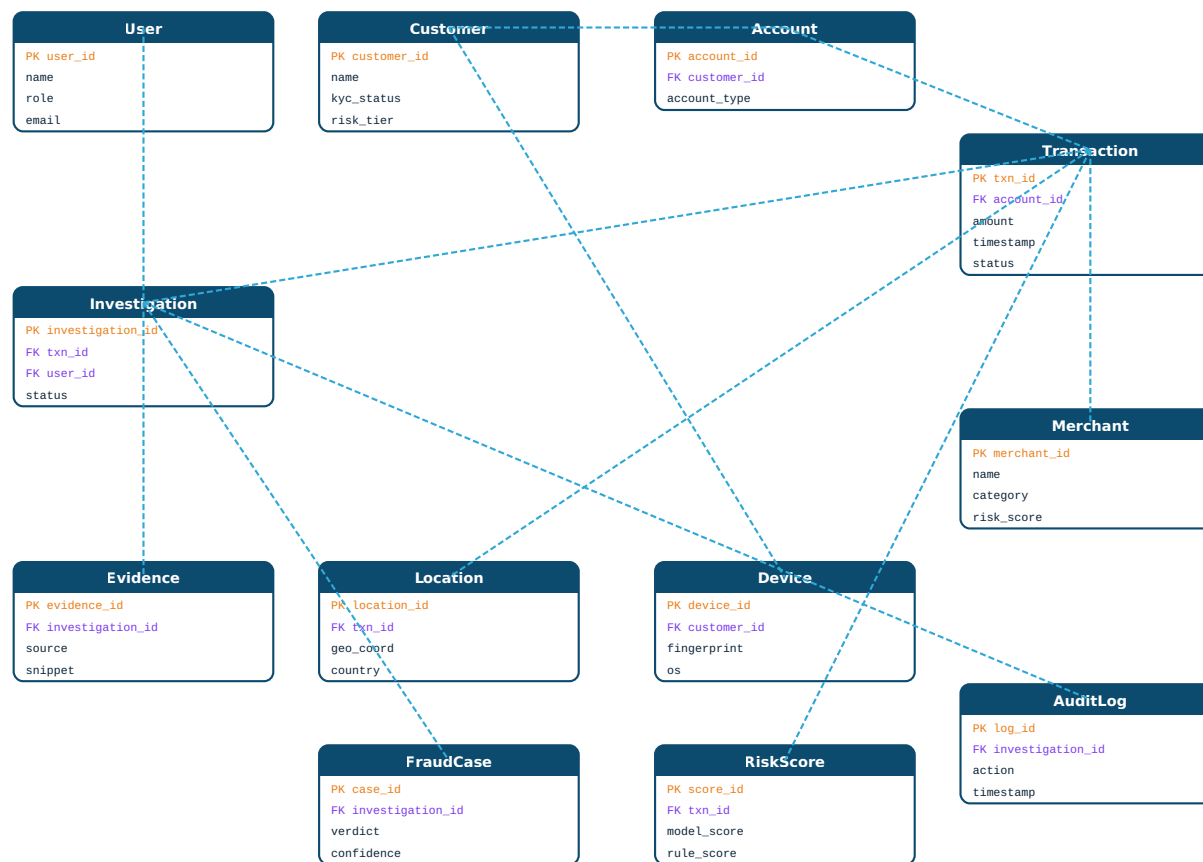


Figure 9.1 — FFIRE Entity Relationship Diagram

## 9.2 Relationship Summary

| Relationship           | Cardinality |
|------------------------|-------------|
| Customer — Account     | One-to-Many |
| Account — Transaction  | One-to-Many |
| Transaction — Merchant | Many-to-One |

|                           |             |
|---------------------------|-------------|
| Customer — Device         | One-to-Many |
| Transaction — Location    | One-to-One  |
| Investigation — Evidence  | One-to-Many |
| Investigation — FraudCase | One-to-One  |
| Investigation — AuditLog  | One-to-Many |

# Database Schema

## 10.1 Core Tables

| Table         | Column           | Type          | Constraint                                   |
|---------------|------------------|---------------|----------------------------------------------|
| customer      | customer_id      | UUID          | PRIMARY KEY                                  |
|               | name             | VARCHAR(150)  | NOT NULL                                     |
|               | kyc_status       | VARCHAR(20)   | NOT NULL                                     |
|               | risk_tier        | VARCHAR(10)   | DEFAULT 'LOW'                                |
| transaction   | txn_id           | UUID          | PRIMARY KEY                                  |
|               | account_id       | UUID          | FOREIGN KEY → account.account_id             |
|               | amount           | DECIMAL(14,2) | NOT NULL                                     |
|               | currency         | CHAR(3)       | NOT NULL                                     |
|               | status           | VARCHAR(20)   | NOT NULL                                     |
| investigation | investigation_id | UUID          | PRIMARY KEY                                  |
|               | txn_id           | UUID          | FOREIGN KEY → transaction.txn_id             |
|               | status           | VARCHAR(20)   | NOT NULL, INDEXED                            |
|               | confidence       | DECIMAL(4,3)  | NULLABLE                                     |
| evidence      | evidence_id      | UUID          | PRIMARY KEY                                  |
|               | investigation_id | UUID          | FOREIGN KEY → investigation.investigation_id |
|               | source           | VARCHAR(50)   | NOT NULL                                     |
|               | snippet          | TEXT          | NOT NULL                                     |
| audit_log     | log_id           | UUID          | PRIMARY KEY                                  |
|               | investigation_id | UUID          | FOREIGN KEY, INDEXED                         |
|               | action           | VARCHAR(50)   | NOT NULL                                     |

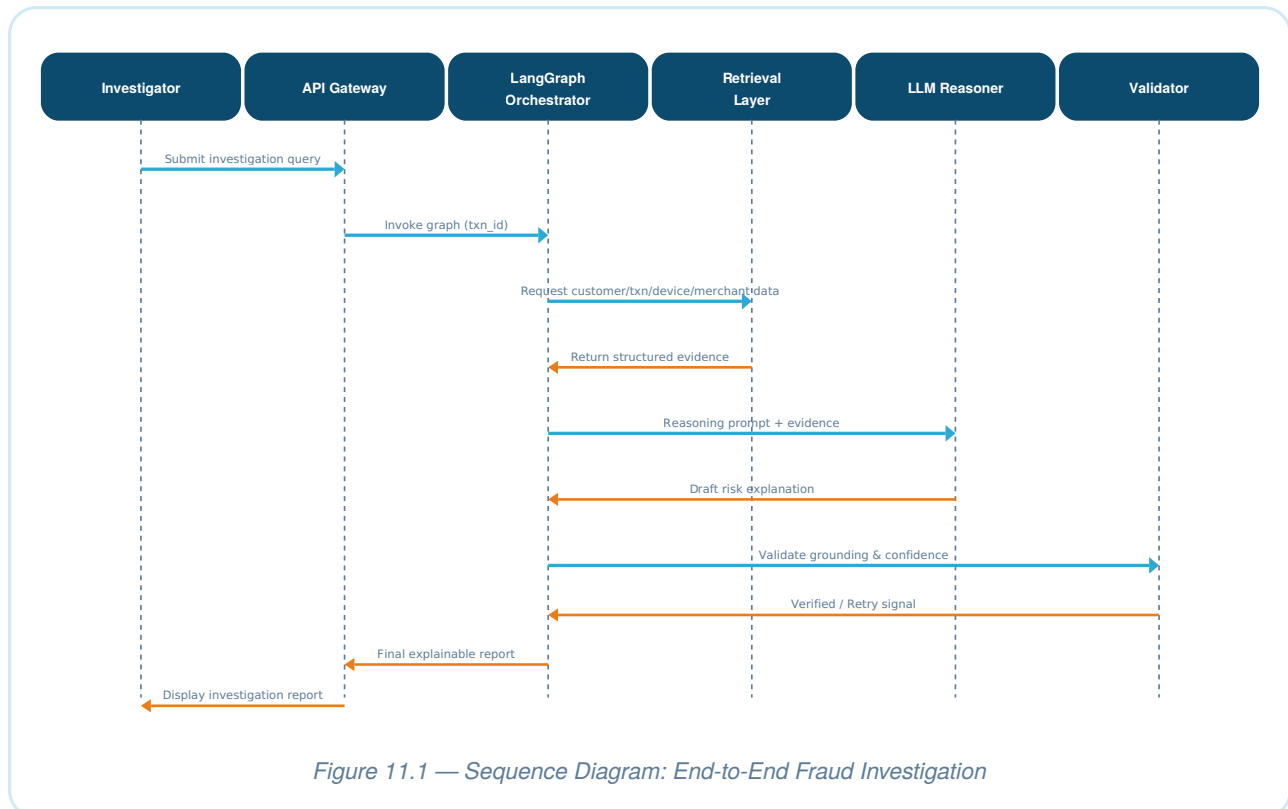
## 10.2 Indexing Strategy

- Composite index on (account\_id, timestamp) for transaction velocity queries.
- B-tree index on investigation.status for dashboard filtering.
- HNSW vector index on the fraud knowledge base embedding column for similarity search.

# Sequence Diagrams

## 11.1 Investigation Request Sequence

The diagram below illustrates the message flow for a single investigation request, from the investigator's submission through to the delivery of the final explainable report.



## 11.2 Retry Sequence (Narrative)

When the Retrieval Layer raises a timeout exception, the Orchestrator catches the failure, increments `state.retry_count`, and re-invokes the failed retrieval node up to a configured maximum (default 3). If retries are exhausted, the Orchestrator routes execution to the Human Review node rather than producing an unsubstantiated report.



# Use Case Diagram

## 12.1 Actors and Use Cases

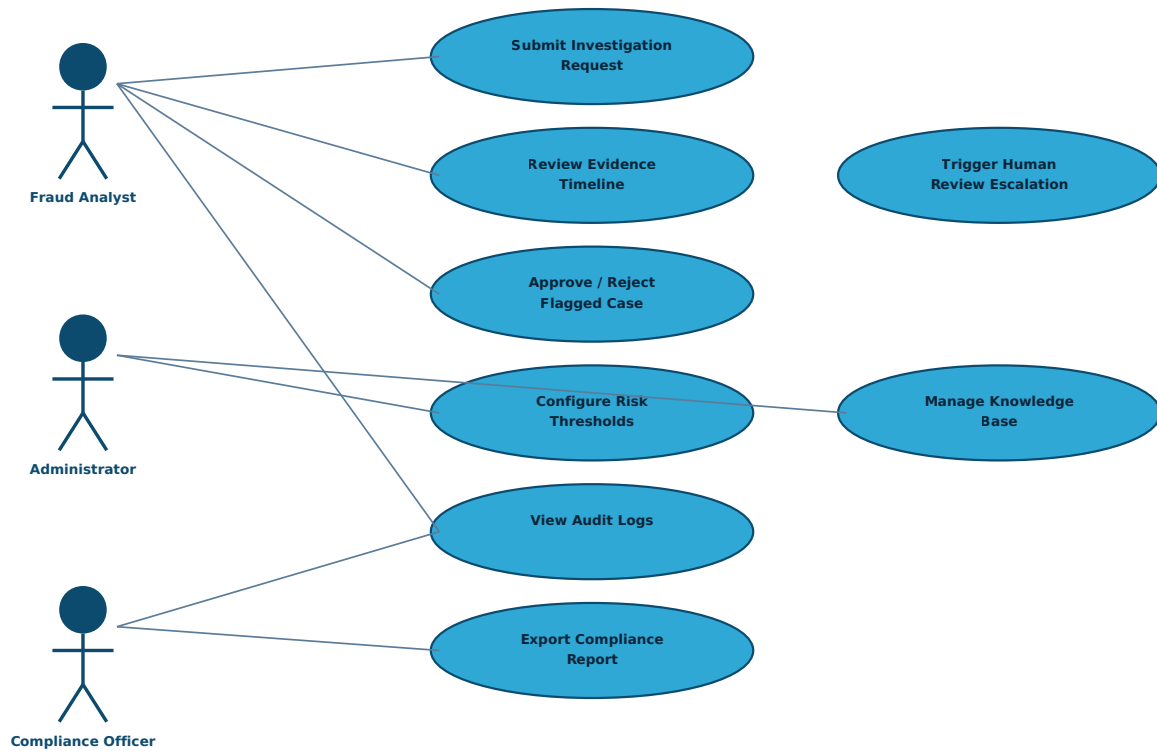


Figure 12.1 — FFIRE Use Case Diagram

## 12.2 Use Case Descriptions

| Use Case                      | Primary Actor      | Description                                                              |
|-------------------------------|--------------------|--------------------------------------------------------------------------|
| Submit Investigation Request  | Fraud Analyst      | Initiate an investigation for a flagged transaction or customer ID.      |
| Review Evidence Timeline      | Fraud Analyst      | Inspect the chronological evidence assembled by the retrieval layer.     |
| Approve / Reject Flagged Case | Fraud Analyst      | Make a final disposition on an investigation escalated for human review. |
| Configure Risk Thresholds     | Administrator      | Adjust confidence and risk-score thresholds used for routing decisions.  |
| View Audit Logs               | Compliance Officer | Review the immutable execution trail of any past investigation.          |

---

Export Compliance Report

Compliance  
Officer

Generate a regulator-ready PDF summary of an investigation.

---

# State Machine Diagram

## 13.1 LangGraph States

The investigation pipeline is implemented as a finite state machine. Transitions are deterministic except for two conditional branches: retry-on-failure and confidence-based human escalation.

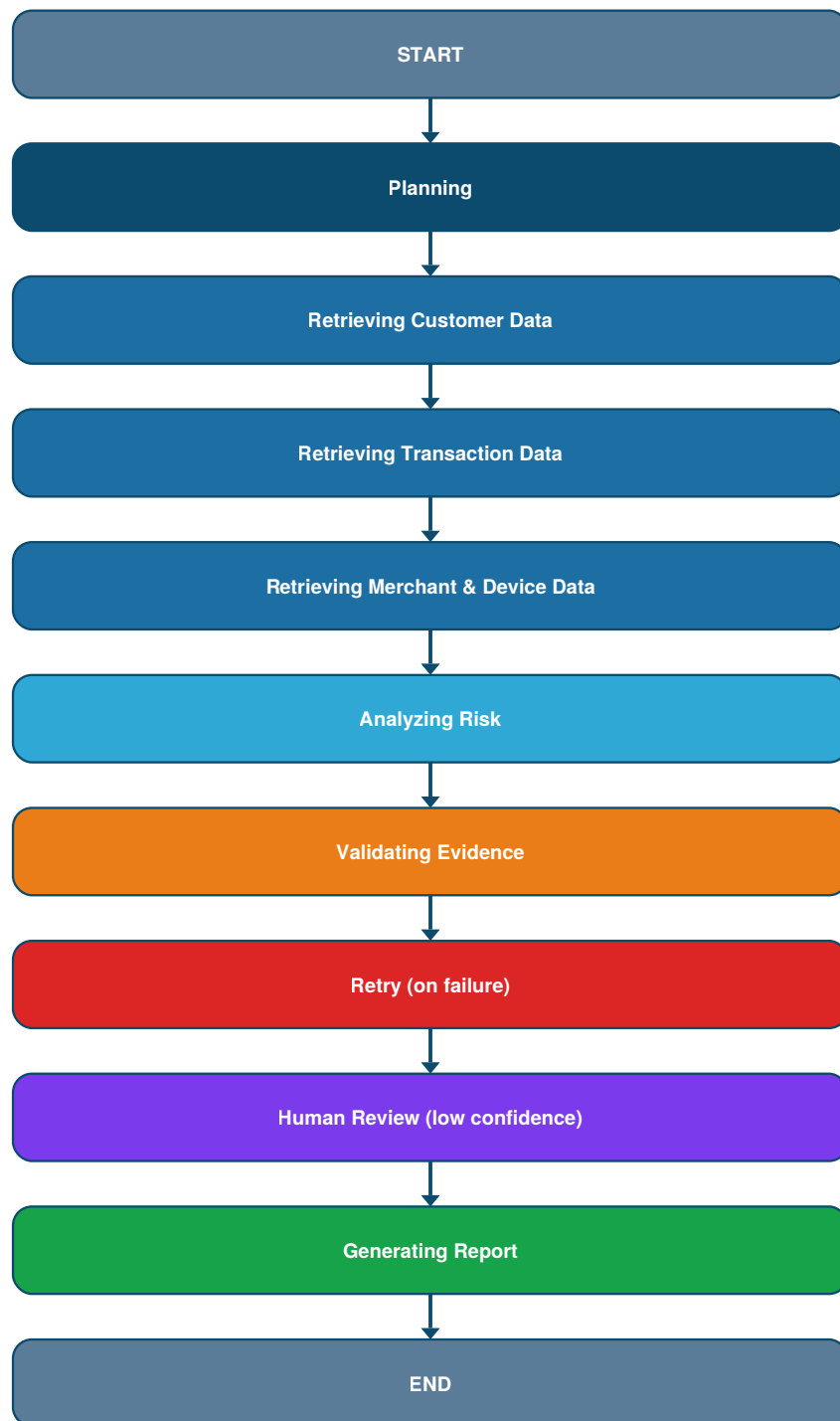


Figure 13.1 — FFIRE State Machine Diagram

## 13.2 Transition Rules

| From State | To State | Condition |
|------------|----------|-----------|
|------------|----------|-----------|

|                     |                                      |                                                     |
|---------------------|--------------------------------------|-----------------------------------------------------|
| Validating Evidence | Retry                                | retrieval failure OR ungrounded claim detected      |
| Validating Evidence | Generating Report                    | all claims grounded AND confidence $\geq$ threshold |
| Validating Evidence | Human Review                         | confidence $<$ threshold after max retries          |
| Retry               | Retrieving Customer/Transaction Data | retry_count $<$ max_retries                         |
| Retry               | Human Review                         | retry_count $\geq$ max_retries                      |

# Activity Diagram

## 14.1 End-to-End Investigation Activity

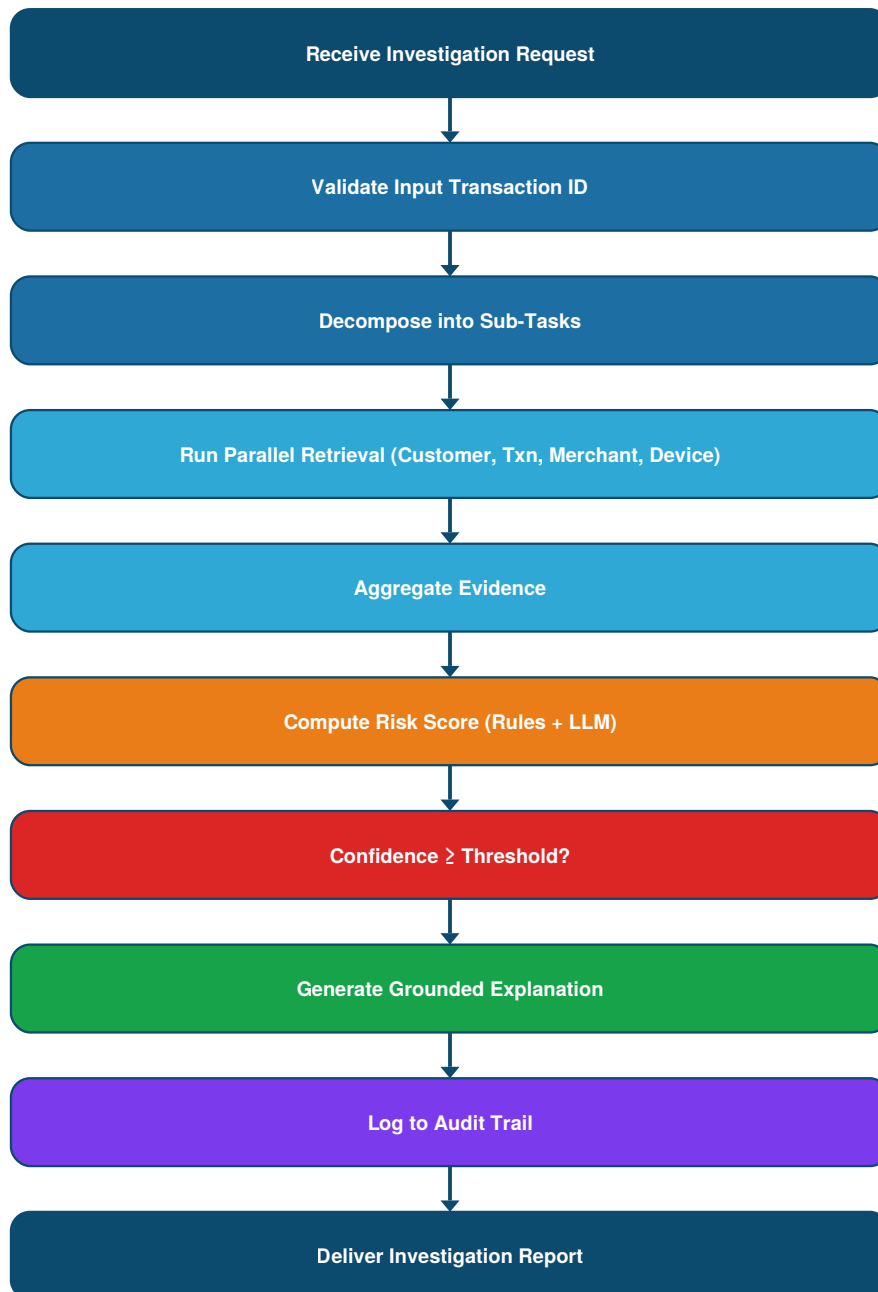


Figure 14.1 — Investigation Activity Diagram

## 14.2 Decision Points

The single decision diamond in the activity flow ("Confidence  $\geq$  Threshold?") determines whether the system proceeds directly to report generation or routes to the retry/human-review sub-flow described in Chapter 13.

# Component Diagram

## 15.1 System Components

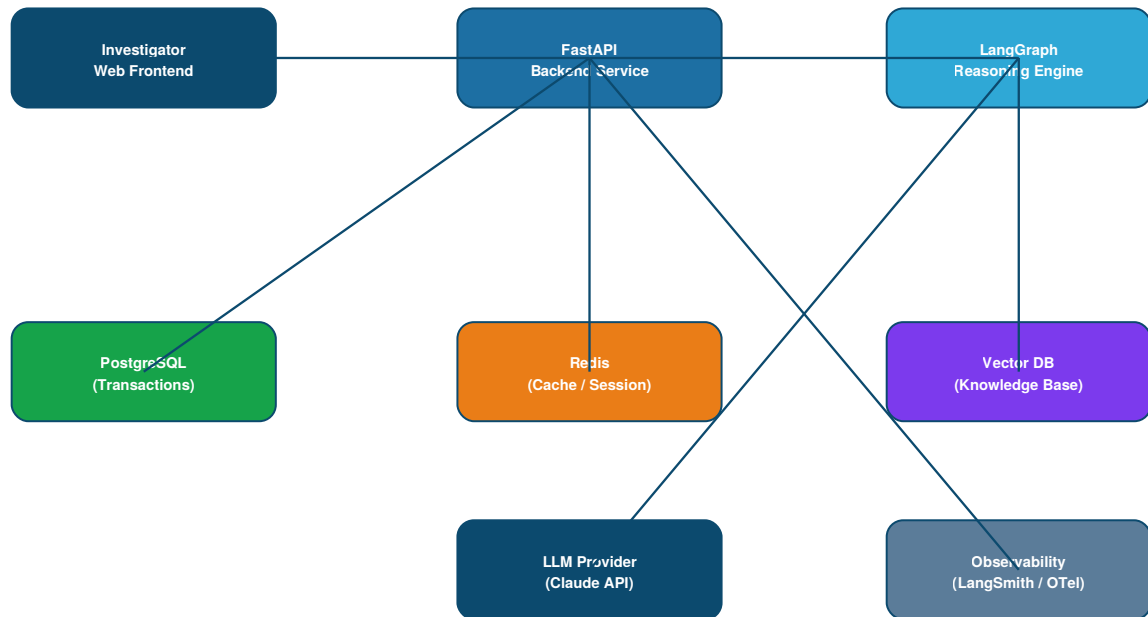


Figure 15.1 — FFIRES Component Diagram

## 15.2 Component Interfaces

| Component                       | Interface                          | Protocol                   |
|---------------------------------|------------------------------------|----------------------------|
| Web Frontend → Backend          | REST /api/v1/investigations        | HTTPS / JSON               |
| Backend → LangGraph Engine      | Internal Python call / async queue | In-process / Redis Streams |
| LangGraph Engine → LLM Provider | Messages API                       | HTTPS / JSON               |
| LangGraph Engine → Vector DB    | Similarity search API              | gRPC / HTTPS               |
| Backend → PostgreSQL            | SQL queries via connection pool    | TCP / TLS                  |



# Deployment Diagram

## 16.1 Deployment Topology

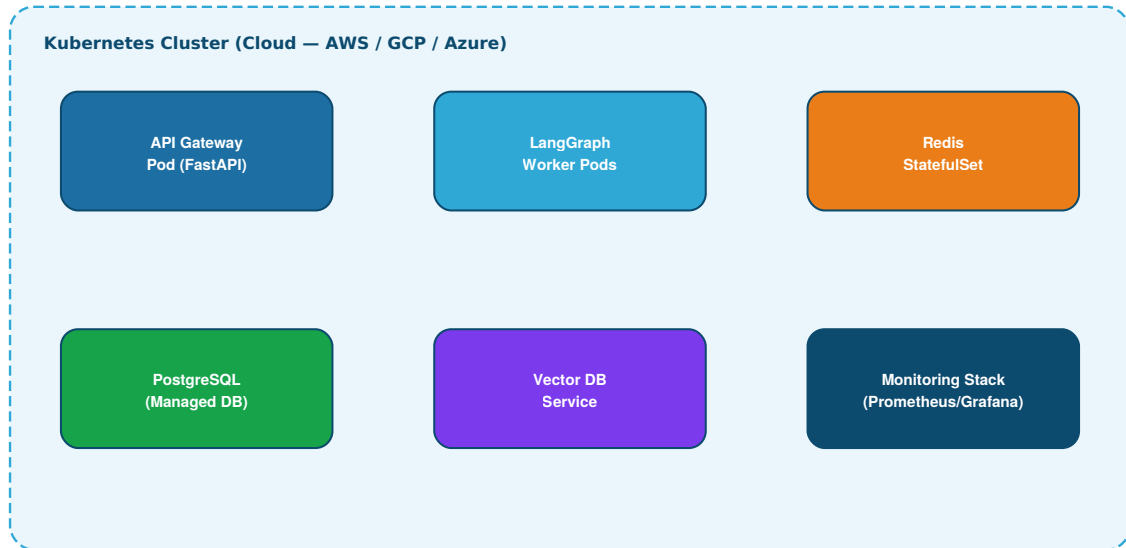


Figure 16.1 — FFIRE Deployment Diagram

## 16.2 Scaling Strategy

- API Gateway and LangGraph Worker pods scale horizontally based on request queue depth.
- PostgreSQL uses a managed read-replica configuration for retrieval-heavy workloads.
- Redis provides session caching and serves as the message bus for asynchronous retry scheduling.
- Monitoring stack tracks node latency, retry counts, and confidence-score distributions in real time.

# Security Architecture

## 17.1 Access Control

FFIRE implements Role-Based Access Control (RBAC) with three primary roles: Fraud Analyst, Administrator, and Compliance Officer, each scoped to a distinct set of permitted operations.

## 17.2 Authentication

All API access requires a signed JWT issued by the identity provider, validated on every request at the API Gateway layer with a maximum token lifetime of 15 minutes and refresh-token rotation.

## 17.3 Data Protection

### Encryption at Rest

AES-256 for all PII and transaction data in PostgreSQL.

### Encryption in Transit

TLS 1.2+ enforced for all internal and external traffic.

### PII Minimization

Only fields required for the current reasoning step are passed into LLM prompts.

### Secrets Management

API keys and database credentials stored in a managed secrets vault, never in source control.

## 17.4 LLM-Specific Guardrails

- Prompt injection defense: retrieved evidence is sanitized and clearly delimited from system instructions.
- Output schema validation: LLM responses must conform to a strict JSON schema before being accepted.
- Rate limiting on LLM and tool invocations to prevent runaway loops.

## 17.5 Audit and Compliance

Every investigation produces an immutable audit record, retained per regulatory data-retention policy, and exportable for examiner review.

# Reasoning Strategy

## 18.1 Reasoning Pipeline

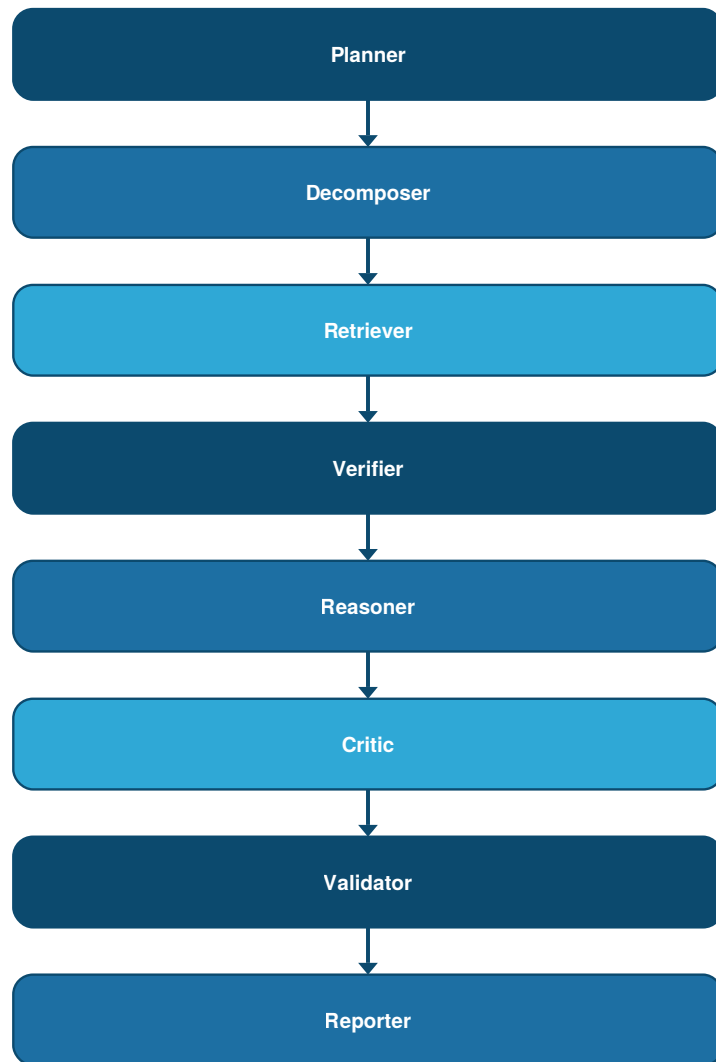


Figure 18.1 — Reasoning Strategy Pipeline

## 18.2 Role of Each Stage

| Stage | Responsibility |
|-------|----------------|
|-------|----------------|

|            |                                                                             |
|------------|-----------------------------------------------------------------------------|
| Planner    | Determines which evidence categories are relevant to the case type.         |
| Decomposer | Breaks the investigation into independently executable sub-tasks.           |
| Retriever  | Executes data retrieval tools against structured and vector sources.        |
| Verifier   | Confirms evidence completeness before reasoning begins.                     |
| Reasoner   | Generates a grounded risk explanation referencing specific evidence fields. |
| Critic     | Reviews the reasoner's draft for unsupported claims or logical gaps.        |
| Validator  | Performs final grounding checks and computes the confidence score.          |
| Reporter   | Formats the final, citation-annotated investigation report.                 |

# Prompt Engineering

## 19.1 Planner Prompt (Template)

You are a financial fraud investigation planner. Given the transaction summary below, output a JSON list of evidence-gathering tasks required to investigate it. Only select tasks from the approved task catalog: [customer\_history, transaction\_detail, merchant\_reputation, device\_fingerprint, location\_check, velocity\_check, historical\_fraud\_match].  
Transaction: {{transaction\_summary}}

## 19.2 Reasoner Prompt (Template)

You are a fraud investigation reasoning engine. Using ONLY the evidence provided below, explain whether this transaction is likely fraudulent. Every claim must reference a specific evidence field by name. If evidence is insufficient to support a conclusion, state this explicitly rather than guessing.  
Evidence: {{evidence\_bundle}}

## 19.3 Validator Prompt (Template)

Review the draft explanation below against the evidence bundle. Flag any sentence that is not directly supported by a named evidence field. Output a list of unsupported claims, or an empty list if all claims are grounded.  
Draft: {{draft\_explanation}}  
Evidence: {{evidence\_bundle}}

## 19.4 Report Prompt (Template)

Format the validated explanation as a structured investigation report with sections: Summary, Evidence Table, Risk Factors, Confidence Score, and Recommendation. Use only the validated, grounded content provided.

# Guardrails

## 20.1 Guardrail Principles

### Grounding

Every claim must cite a specific evidence field; ungrounded claims are rejected before reaching the report.

### Source Verification

Evidence must originate from an authenticated retrieval tool call, never from LLM free recall.

### Confidence Threshold

Reports are only finalized automatically when confidence  $\geq 0.85$  (configurable).

### Retry

Transient failures trigger automatic retry up to 3 attempts before escalation.

### Escalation

Low-confidence or high-risk cases are routed to human review rather than auto-finalized.

### Hallucination Detection

A dedicated Critic node cross-checks every sentence of the draft explanation against the evidence bundle.

## 20.2 Guardrail Enforcement Flow

The Validator node parses the Reasoner's draft output, extracts each factual claim, and checks it against the evidence bundle's field names and values. Any claim that cannot be matched to a specific evidence field is flagged; if any flagged claim exists, the graph transitions to the Retry state rather than producing a final report, ensuring the system fails closed rather than open.

# Risk Scoring

## 21.1 Composite Risk Model

FFIRE computes a composite risk score combining three independent signals, weighted and normalized to a 0–100 scale.

| Signal                   | Weight | Source                                                          |
|--------------------------|--------|-----------------------------------------------------------------|
| Rule Engine Score        | 35%    | Velocity limits, geolocation mismatch, device-change heuristics |
| LLM Reasoning Score      | 40%    | Grounded explanation confidence from the Reasoner/Critic stages |
| Historical Pattern Match | 25%    | Vector similarity to confirmed historical fraud cases           |

## 21.2 Risk Tiers

| Score Range | Tier   | Action                                        |
|-------------|--------|-----------------------------------------------|
| 0–39        | Low    | Auto-clear, log only                          |
| 40–69       | Medium | Auto-generate report, periodic analyst review |
| 70–100      | High   | Mandatory human review before disposition     |

# API Specification

## 22.1 Create Investigation

POST /api/v1/investigations

```
{
  "transaction_id": "T-78491",
  "priority": "standard"
}
```

Response 202 Accepted

```
{
  "investigation_id": "INV-20260630-0091",
  "status": "PROCESSING"
}
```

## 22.2 Get Investigation Report

GET /api/v1/investigations/{investigation\_id}

Response 200 OK

```
{
  "investigation_id": "INV-20260630-0091",
  "status": "COMPLETED",
  "risk_score": 82,
  "confidence": 0.91,
  "summary": "Transaction flagged due to new device, location mismatch...",
  "evidence": [
    {"field": "device.first_seen", "value": "2026-06-30"},
    {"field": "location.country_diff", "value": true}
  ]
}
```

## 22.3 Error Codes

| Code | Meaning                                                       |
|------|---------------------------------------------------------------|
| 400  | Invalid or malformed transaction identifier                   |
| 404  | Investigation or transaction not found                        |
| 409  | Investigation already in progress for this transaction        |
| 422  | Insufficient evidence to complete investigation automatically |
| 503  | Upstream retrieval or LLM provider unavailable                |



# Frontend Design

## 23.1 Dashboard Overview

The investigator dashboard surfaces a queue of pending and completed investigations, filterable by risk tier, status, and assigned analyst.

## 23.2 Key Screens

| Screen               | Purpose                                                                      |
|----------------------|------------------------------------------------------------------------------|
| Transaction Search   | Locate a transaction or customer to initiate an investigation.               |
| Fraud Timeline       | Chronological view of evidence events leading up to the flagged transaction. |
| Evidence Panel       | Inspect raw evidence records underlying each cited claim.                    |
| Reasoning Graph View | Visualize the LangGraph execution trace for the current investigation.       |
| Investigation Report | Final, citation-annotated explanation with risk tier and recommendation.     |

## Future Scope

- **Real-Time Streaming:** Move from on-demand investigation to continuous streaming risk assessment as transactions occur.
- **Graph Database Integration:** Model customer-merchant-device relationships as a graph for advanced ring-fraud detection.
- **Cross-Bank Fraud Intelligence:** Federated sharing of anonymized fraud patterns across institutions.
- **Multi-Agent Collaboration:** Specialized sub-agents for card fraud, wire fraud, and account-takeover investigations operating under a shared orchestration layer.
- **Federated Learning:** Improve the rule and embedding models using decentralized training across participating institutions without sharing raw customer data.
- **Voice Investigator Interface:** Conversational access to investigation reports for field analysts.

## Conclusion

FFIRE demonstrates that financial fraud investigation can be automated without sacrificing explainability, determinism, or auditability — the three properties most critical to regulated financial environments. By modelling the investigation as an explicit LangGraph state machine with strict grounding guardrails, retry logic, and confidence-based human escalation, the system delivers investigation-grade explanations rather than opaque risk scores. This architecture generalizes beyond fraud investigation to any enterprise domain requiring auditable, multi-step AI reasoning — including IT incident root-cause analysis, manufacturing defect investigation, and regulatory compliance review — positioning FFire as a reusable reasoning platform rather than a single-purpose application.

# Glossary, Abbreviations & References

## A.1 Glossary

### LangGraph

A library for building stateful, multi-step LLM applications as explicit execution graphs.

### Grounding

The property that every generated claim is traceable to a specific piece of retrieved evidence.

### RAG

Retrieval-Augmented Generation — supplementing LLM context with retrieved external data.

### Guardrail

A validation mechanism that prevents the system from producing unsupported or unsafe output.

### KYC

Know Your Customer — identity verification processes required of financial institutions.

### AML

Anti-Money Laundering — regulatory framework for detecting and reporting financial crime.

### Confidence Score

A normalized value representing the system's certainty in a generated conclusion.

### State Machine

A computational model in which the system transitions between a finite set of well-defined states.

## A.2 Abbreviations

| Abbreviation | Full Form                                      |
|--------------|------------------------------------------------|
| FFIRE        | Financial Fraud Investigation Reasoning Engine |
| SRS          | Software Requirements Specification            |
| DFD          | Data Flow Diagram                              |
| ER           | Entity Relationship                            |
| RBAC         | Role-Based Access Control                      |
| JWT          | JSON Web Token                                 |
| API          | Application Programming Interface              |
| LLM          | Large Language Model                           |

## A.3 References

1. LangChain / LangGraph official documentation — workflow orchestration patterns for LLM applications.
2. Anthropic, "Building Effective AI Agents" — design patterns for grounded, multi-step agent systems.
3. FATF (Financial Action Task Force) — international standards for anti-money-laundering controls.
4. NIST AI Risk Management Framework — guidance on explainability and trustworthy AI systems.
5. OWASP Top 10 for LLM Applications — prompt injection and LLM-specific security guidance.

End of Document — FFIRE Software Requirements Specification v1.0 | Confidential